

Tareas Unidad 6

Desarrollo Web en Entorno Servidor

Autor: Derimán Tejera Fumero

Fecha: 16/11/2024

Índice

Enunciado de la práctica	3
Explicación de la resolución de la práctica	6
Descarga e instalación de Composer	6
Cargando las tablas y datos en la BD	6
Inicializando composer desde Visual Studio Code	10
Creamos las clases en la carpeta src	14
Realizando las pruebas con SoapUI 5.7.2 SIN WSDL	19
Solicitud XML para GetPVPRequest	21
Solicitud XML para GetStockRequest	22
Solicitud XML para GetFamiliasRequest	23
Solicitud XML para GetProductosFamiliasRequest	24
Realizando las pruebas con SoapUI 5.7.2 CON WSDL	25
Solicitud XML para GetPVP	28
Solicitud XML para GetStock	29
Solicitud XML para GetFamilias	30
Solicitud XML para GetProductosFamilia	31
Servicio de prueba clienteW.php	32
Generación automática de clases	33
Servicio de prueba clienteW2.php	36

Enunciado de la práctica

Vamos a seguir utilizando para esta tarea la base de datos correspondiente a la tienda web.

En primer lugar, deberás crear una base de datos de nombre **"tarea6"**, similar a la que usamos en los ejercicios anteriores. Para acceder puedes reutilizar el usuario **"gestor"** y su contraseña **"secreto"**.

A continuación, utilizarás **"PHP SOAP"** para crear un servicio web con cuatro funciones que expongan información de la base de datos de la tienda online. En el primer servidor no deberás utilizaremos ningún archivo **WSDL**. Después generaremos uno y lo usaremos.

Échale un vistazo a la estructura de la práctica propuesta para que te hagas una primera idea de los archivos que se piden.

Para la base de datos se deja en un archivo comprimido los archivos para crear la base de datos **"tarea6"** y sus tablas, y el archivo para guardar datos de ejemplos para las tablas: [Archivos](#) (zip - 8392 B)

Baseytablastarea6.sql:

```
1 -- 1.- Creamos la Base de Datos
2 create database tarea6 DEFAULT CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
3 -- Seleccionamos la base de datos "tarea6"
4 use tarea6;
5 -- 2.- Creamos las tablas
6 -- 2.1.1.- Tabla tienda
7 create table if not exists tiendas(
8     id int auto_increment primary key,
9     nombre varchar(100) not null,
10    tlf varchar(13) null
11 );
12 -- 2.1.2.- Tabla familia
13 create table if not exists familias(
14     cod varchar(6) primary key,
15     nombre varchar(200) not null
16 );
17 -- 2.1.3.- Tabla producto
18 create table if not exists productos(
19     id int auto_increment primary key,
20     nombre varchar(200) not null,
21     nombre_corto varchar(50) unique not null,
22     descripcion text null,
23     pvp decimal(10, 2) not null,
24     familia varchar(6) not null,
25     constraint fk_prod_fam foreign key(familia) references familias(cod) on update cascade on delete cascade
26 );
27 -- 2.1.4 Tabla stocks
28 create table if not exists stocks(
29     producto int,
30     tienda int,
31     unidades int unsigned not null,
32     constraint pk_stock primary key(producto, tienda),
33     constraint fk_stock_prod foreign key(producto) references productos(id) on update cascade on delete cascade,
34     constraint fk_stock_tienda foreign key(tienda) references tiendas(id) on update cascade on delete cascade
35 );
36 );
37 -- 3.- Creamos un usuario descomentalo si lo necesitas
38 -- create user gestor@'localhost' identified by "secreto";
39 -- 4.- Le damos permiso en la base de datos "tarea6"
40 grant all on tarea6.* to gestor@'localhost';
```

datostarea6.sql:

(archivo demasiado grande para poder mostrarlo, consultar directamente).

Tendremos las carpetas:

- **"public"**: donde crearemos los ficheros para llamar a los servicios que crearemos.
- **"servidorSoap"**: donde generaremos nuestros servicios.
- **"src"**: Guardaremos todas nuestras clases, la clase **"Operaciones"** con todas las funciones que nos ofrecerá, la clase **"Conexion"**, y las clases **"Producto, Familia y Stock"**, con los métodos para acceder a la bases de datos y gestionar las tablas, como vimos en unidades anteriores.
- **"vendor"**: La generará automáticamente **Composer**.

Para esta práctica se pedirá **"autoload"** de las clases, **"php2wsdl"** y **"wsdl2phpgenerator"**, todo instalado con **"Composer"**

- Crearemos el servidor **SOAP "servicio.php"** en la carpeta **"servidorSoap"**. Este servicio no tendrá asociado ningún archivo **WSDL** y ofrecerá las funciones siguientes (Todas implementadas en **"src/Operaciones.php"**):
 - **getPVP**. Esta función recibirá como parámetro el código de un producto, y devolverá el PVP correspondiente al mismo.
 - **getStock**. Esta función recibirá dos parámetros: el código de un producto y el código de una tienda. Devolverá el **stock** existente en dicha tienda del producto.
 - **getFamilias**. No recibe parámetros y devuelve un array con los códigos de todas las familias existentes.
 - **getProductosFamilia**. Recibe como parámetro el código de una familia y devuelve un array con los códigos de todos los productos de esa familia.
- Para comprobar la correcta ejecución del servicio, programa también un cliente con nombre **"cliente.php"** en la carpeta **"public"** que realice una llamada a cada una de las funciones programadas y muestre el resultado obtenido.
- Con la extensión **"php2wsdl"** crea el archivo **"public/generarWsdl.php"** para generar el documento **WSDL "servicio.wsdl"** que guardaremos en **"servidorSoap"**. Utilizando este documento nos crearemos un nuevo servidor **SOAP, "servicioW.php"** que lo utilice.
- Para comprobar la correcta ejecución de este nuevo servicio, programa también un cliente con nombre **"clienteW.php"** en la carpeta **"public"** que realice una llamada a cada una de las funciones programadas y muestre el resultado obtenido.
- Partiendo de este nuevo servicio y de su descripción crearemos el fichero **"public/generarClases"**, que utilizará la herramienta **"wsdl2php"** para obtener una clase **PHP**. Esta clase la guardaremos en la carpeta **"Clases1"** dentro de **"src"**.

- Crea un nuevo cliente llamado "**clienteW2.php**" que se base en ésta clase, para probar el nuevo servicio, mostrando los resultados obtenidos de forma similar a como hiciste en los casos anteriores.

Explicación de la resolución de la práctica

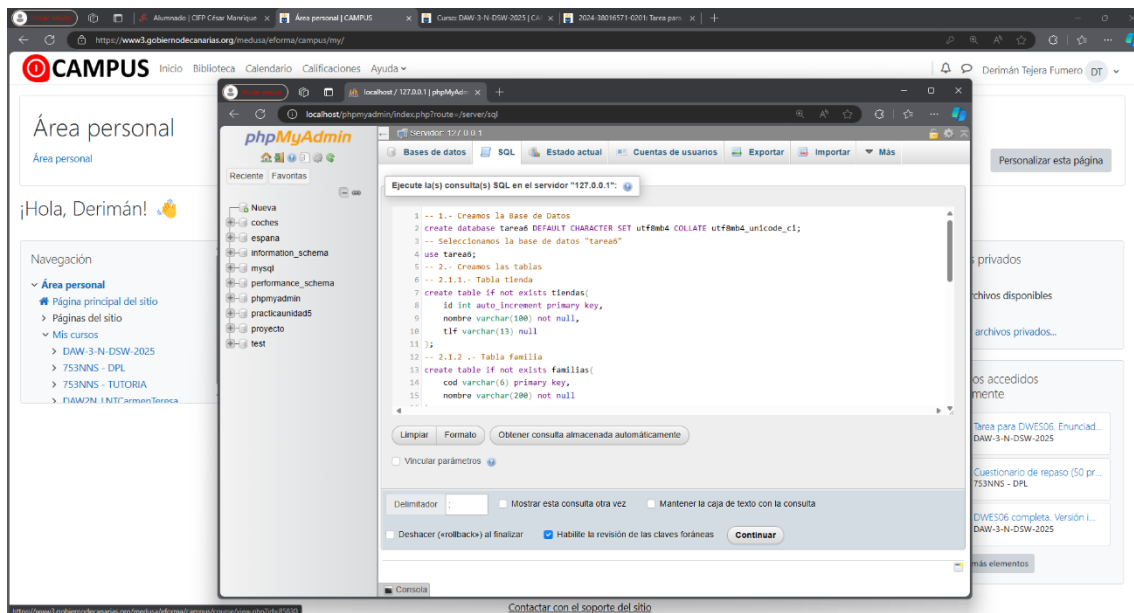
Descarga e instalación de Composer

En primer lugar, descargamos composer desde este [enlace](#). Se siguen los pasos indicados por la instalación.

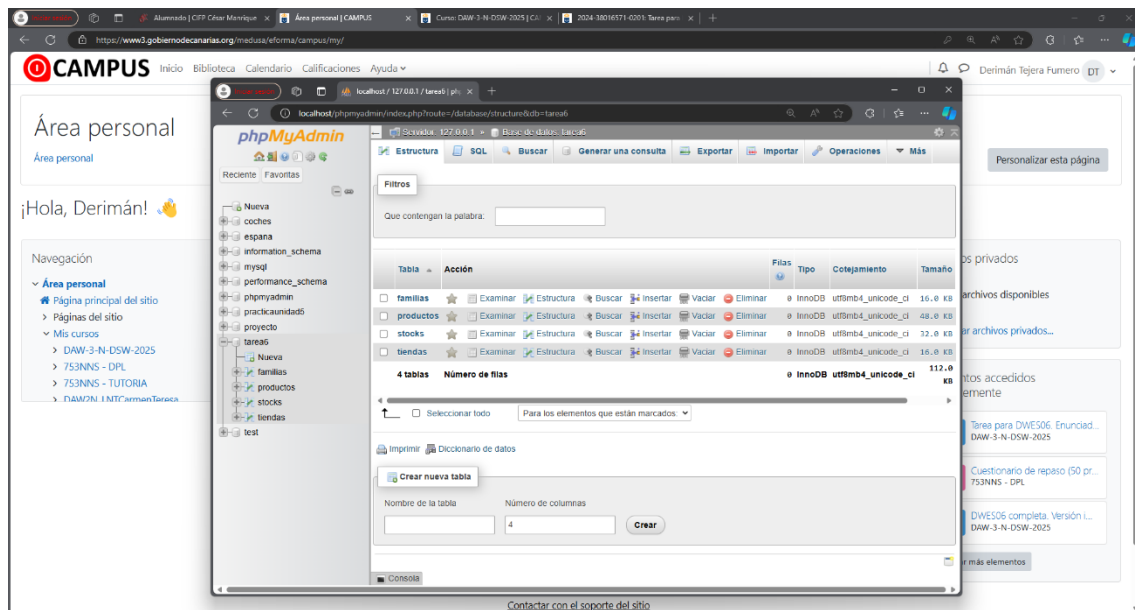
Cargando las tablas y datos en la BD

En primer lugar, descargamos composer desde este [enlace](#). Se siguen los pasos indicados por la instalación.

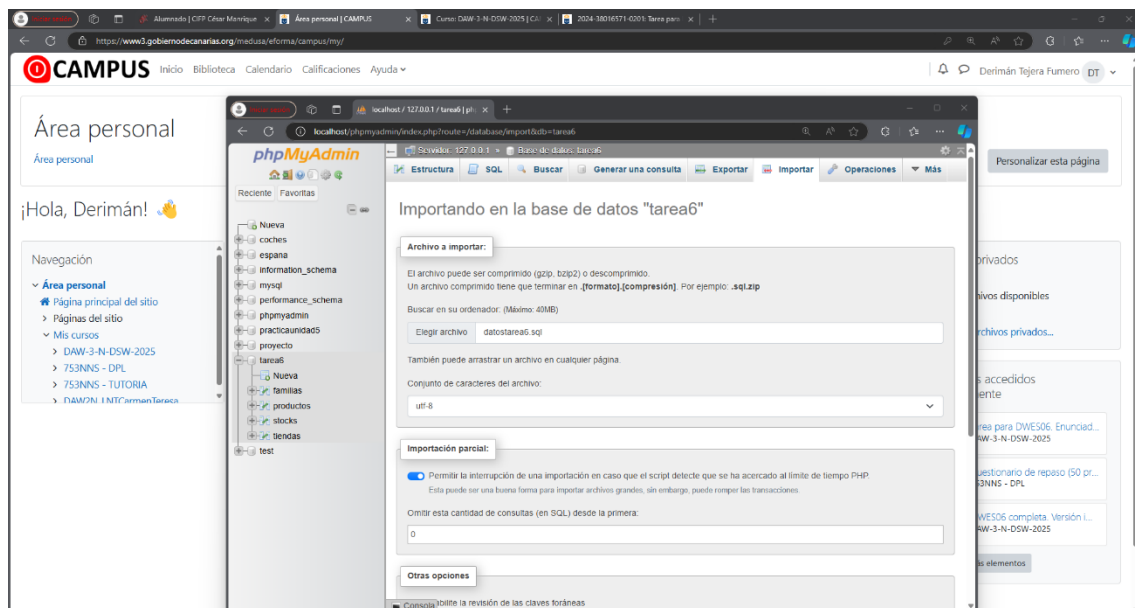
Sin seleccionar ninguna base de datos de las ya existentes (en el lado izquierdo), entramos en la pestaña superior de “SQL” y copiamos el contenido del archivo **Baseytablastarea6.sql** y luego pulsamos el botón de la parte inferior “Continuar”.

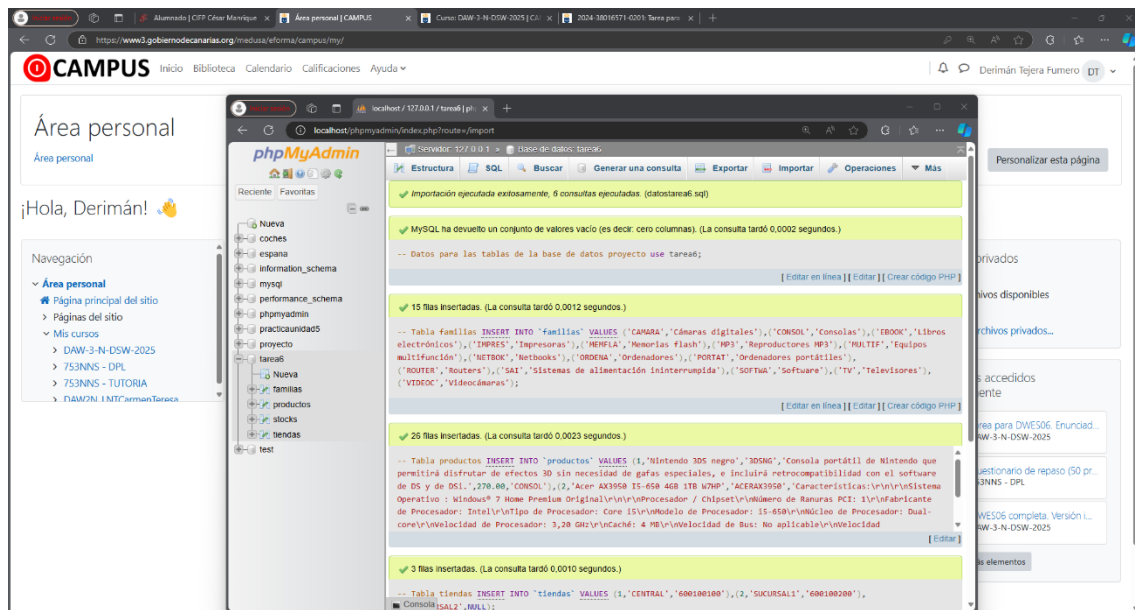


El resultado de cargar el código del archivo es la creación de las tablas y sus relaciones:

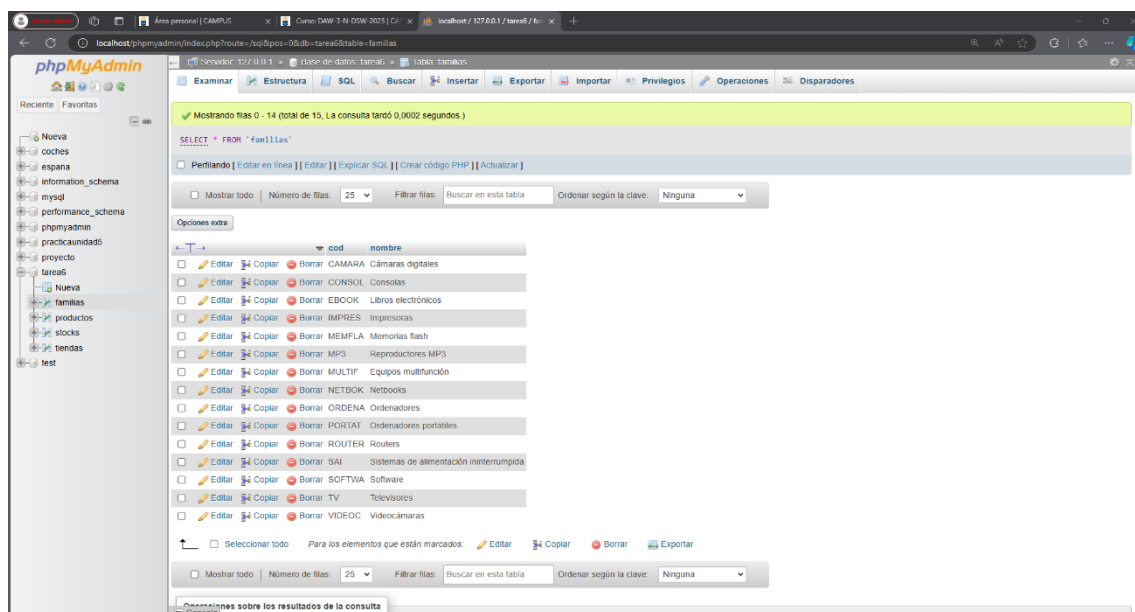


Ahora cargaremos los datos en las tablas, para ello, seleccionaremos la pestaña “Importar” y seleccionamos el archivo que los contiene, **datostarea6.sql**:





Una vez finalizado todo el proceso, creadas las tablas y llenadas con los datos, la base de datos se vería así:



phpMyAdmin interface showing the 'productos' table. The table contains 10 rows of product data, including details like name, description, price, and category.

id	nombre	nombre_corto	descripcion	precio	familia
1	Nintendo 3DS negro	3DSNG	Consola portátil de Nintendo que permitirá distri...	270.00	CONSOL
2	Acer AX3950 i5-550 4GB 1TB W7HP	ACERAX3950	Sistema Operativo: Windows® 7...	410.00	ORDENA
3	Archos Clipper MP3 2GB negro	ARCLPMP32GBN	Almacenamiento interno Disponi...	26.70	MP3
4	Sony Bravia 32IN FULLHD KDL-32EX400	BRAVIA26EX400	Full HD: Vea deportes película...	356.90	TV
5	Asus EEEPC 100SPX N455 1 250 BL	EEEPC100SPX	Características: Procesador: 1660 MHz, N455, Inte...	245.40	NETBOK
6	HP Mini 110-3120 10.1LED N455 1GB 250GB W7S negro	HPMINI103120	Características: Sistema operativo instalado Wi...	270.00	NETBOK
7	Canon Ixus 119HS azul	IXUS119HSBZ	Características: HS System (12.1 MP) Zoom 4x. 2...	196.70	CAMARA
8	Kingston DataTraveler 16GB DT101G2 USB2.0 negro	KSTD101G2	Características: Capacidades: 16GB Dimensiones...	19.20	MEMFLA
9	Kingston DataTraveler G3 32GB rojo	KSTDTC32GBR	Características: Tipo de producto: Unidad flash ...	40.00	MEMFLA
10	Kingston MicroSDHC 8GB	KSTMSDHC8GB	Kingston tarjeta de memoria flash 8 GB microSDHC	10.20	MEMFLA

phpMyAdmin interface showing the 'stocks' table. The table contains 14 rows of stock data, including details like product name, store, and units.

producto	tienda	unidades
1	1	2
1	2	1
2	1	1
3	2	1
3	3	2
4	3	1
5	1	2
5	2	1
6	2	1
6	3	2
7	2	2
8	3	1
9	1	1
9	2	2
10	2	2
10	3	2
11	2	1
12	1	1
13	2	2
14	1	3

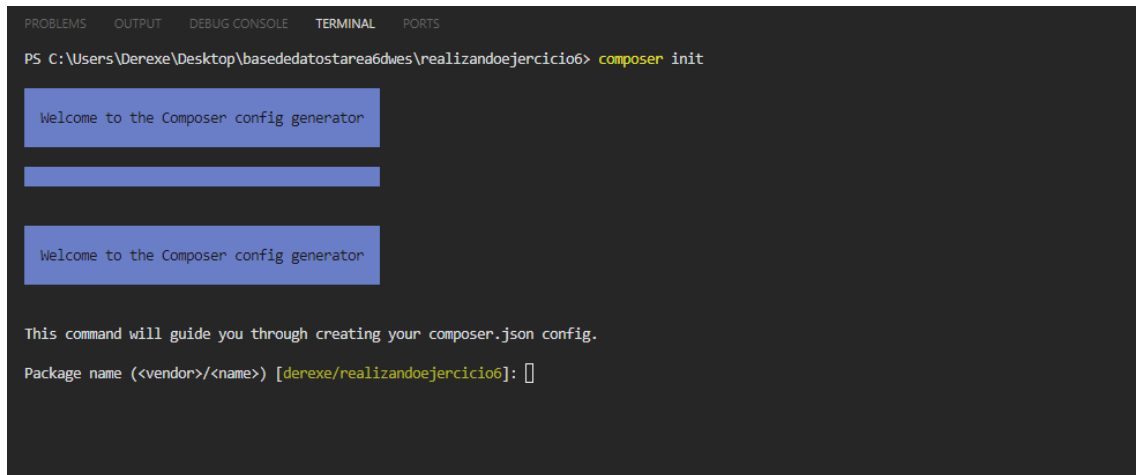
phpMyAdmin interface showing the 'tiendas' table. The table contains 3 rows of store data, including details like store name and address.

id	nombre	tit
1	CENTRAL	600100100
2	SUCURSAL1	600100200
3	SUCURSAL2	NULL

Inicializando composer desde Visual Studio Code

Creamos una carpeta para el proyecto y la abrimos en VS Code. Iniciamos el terminal en con CTRL+ñ o desde View > Terminal.

Inicializamos composer con el comando **composer init**:



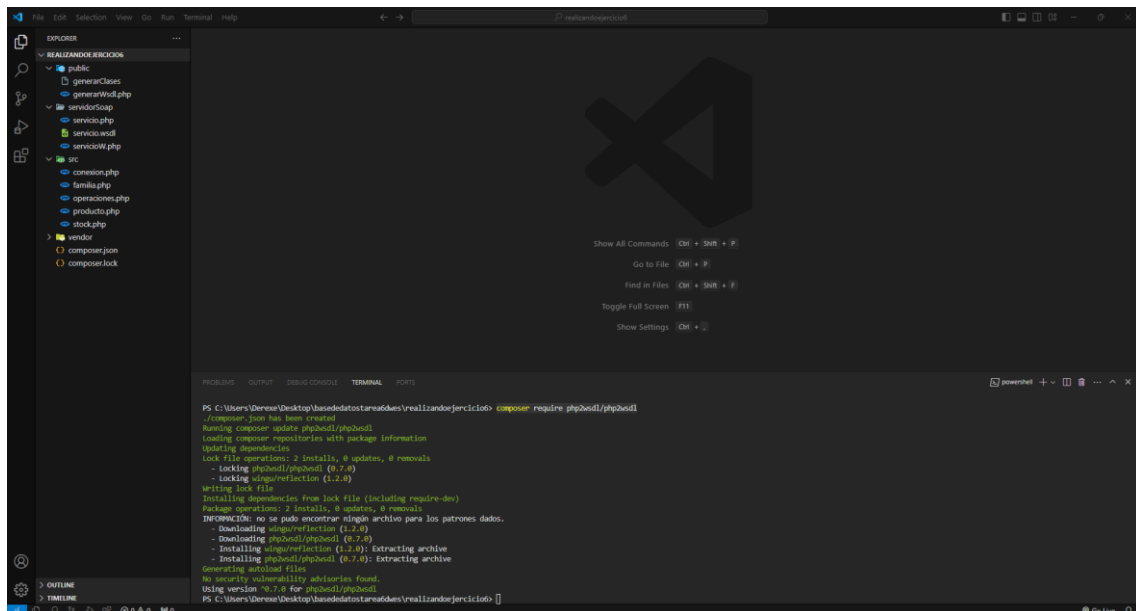
```
PS C:\Users\Derexe\Desktop\basededatostarea6dws\realizandoejercicio6> composer init

Welcome to the Composer config generator

Package name (<vendor>/<name>) [derexe/realizandoejercicio6]:
```

Ahora instalamos las dependencias necesarias:

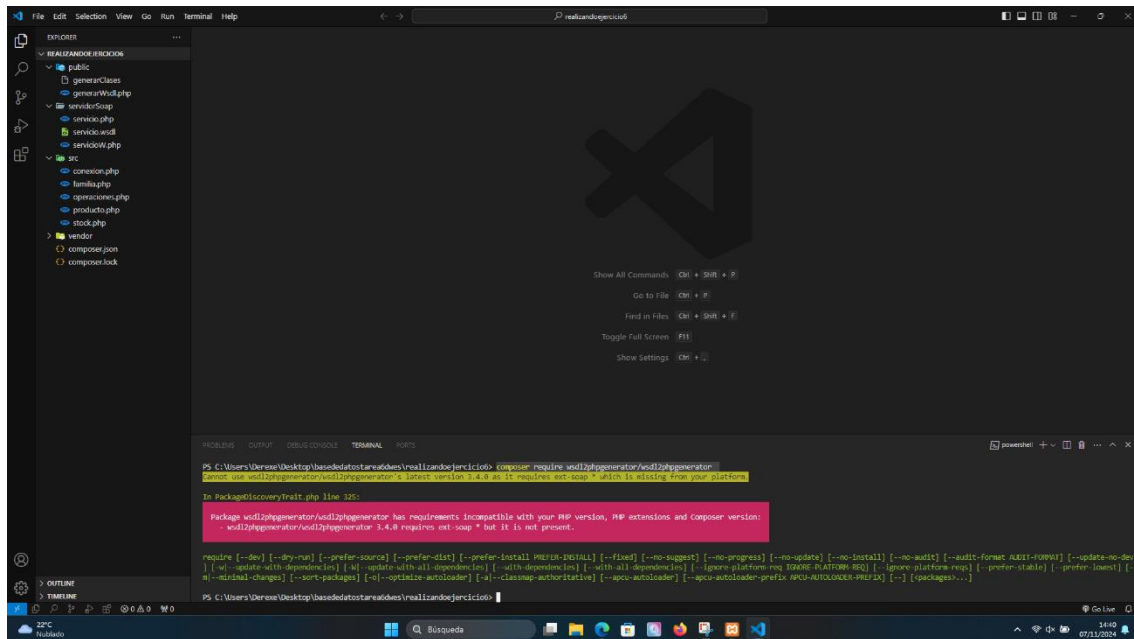
Comenzamos con php2wsdl con el comando: **composer require php2wsdl/php2wsdl**:



```
PS C:\Users\Derexe\Desktop\basededatostarea6dws\realizandoejercicio6> composer require php2wsdl/php2wsdl
./composer.json has been created
Running composer update php2wsdl/php2wsdl
Loading composer repositories with package information
Updating dependencies
Lock file operations: 2 installs, 0 updates, 0 removals
  - Locking php2wsdl/php2wsdl (0.7.0)
  - Locking singularreflection (1.2.0)
Writing lock file
Installing dependencies from lock file (including require-dev)
Package operations: 2 installs, 0 updates, 0 removals
  -> Downloading singularreflection (1.2.0)
  -> Downloading php2wsdl/php2wsdl (0.7.0)
  -> Installing singularreflection (1.2.0): Extracting archive
  -> Installing php2wsdl/php2wsdl (0.7.0): Extracting archive
Generating autoload files
No security vulnerability advisories found.
Using version ^0.7.0 for php2wsdl/php2wsdl
PS C:\Users\Derexe\Desktop\basededatostarea6dws\realizandoejercicio6>
```

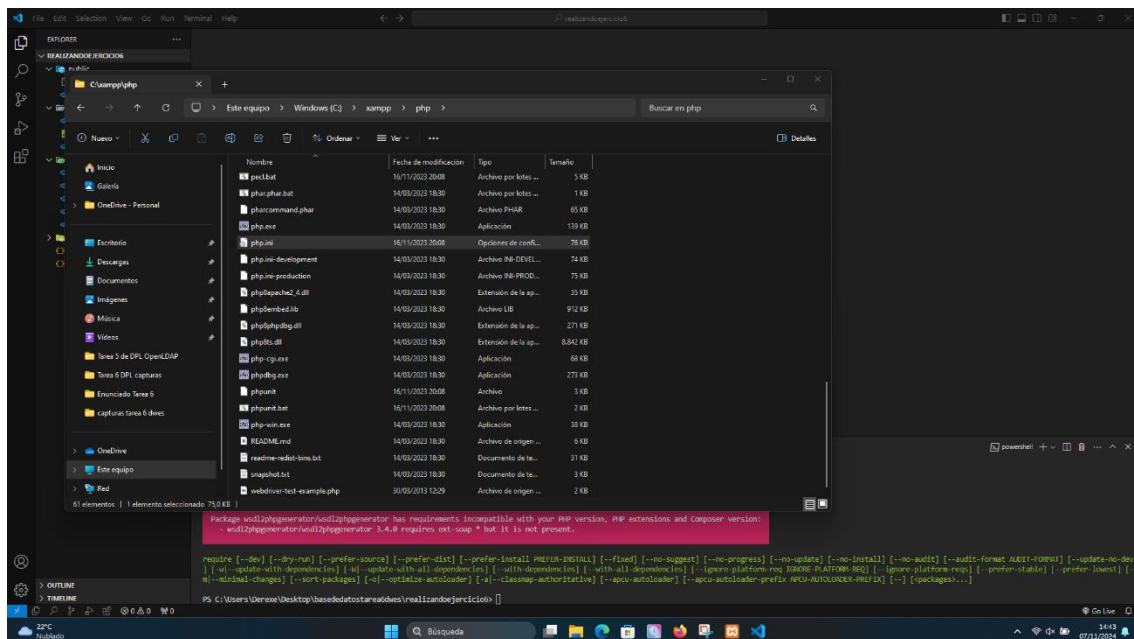
Como se puede ver, se ha instalado correctamente.

Continuamos con wsdl2phpgenerator con el comando: **composer require wsdl2phpgenerator/wsdl2phpgenerator**.

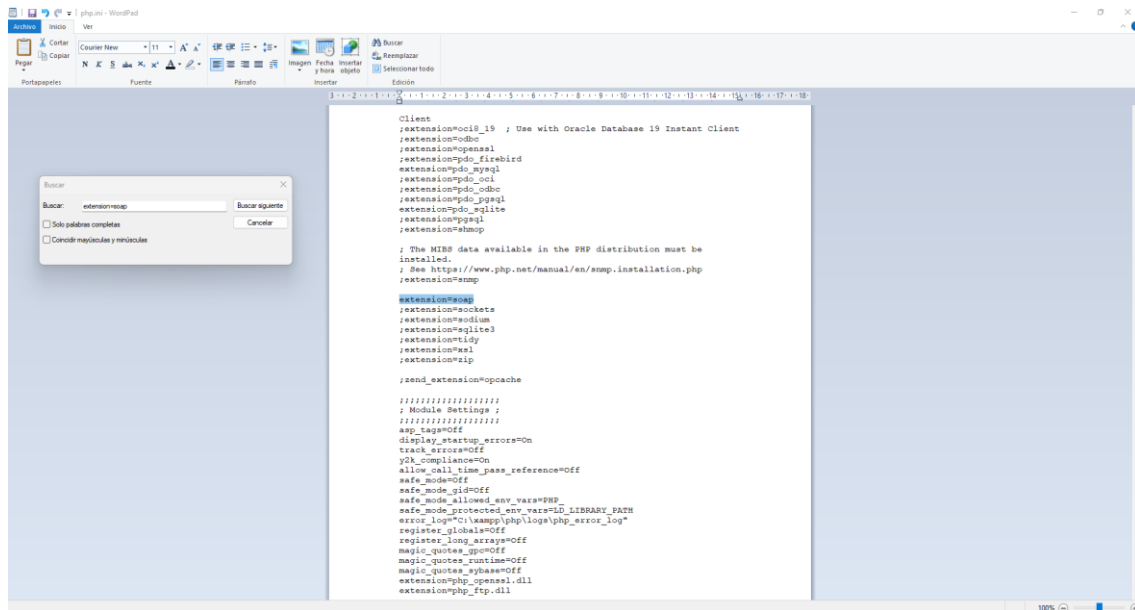


Como puede verse no ha sido posible debido a un error.

Para resolver este error, debemos acceder al archivo php.ini:



Buscamos la línea en el archivo que contiene **extensión=soap** y la descomentamos eliminando el **;**.



Guardamos el cambio y salimos del archivo.

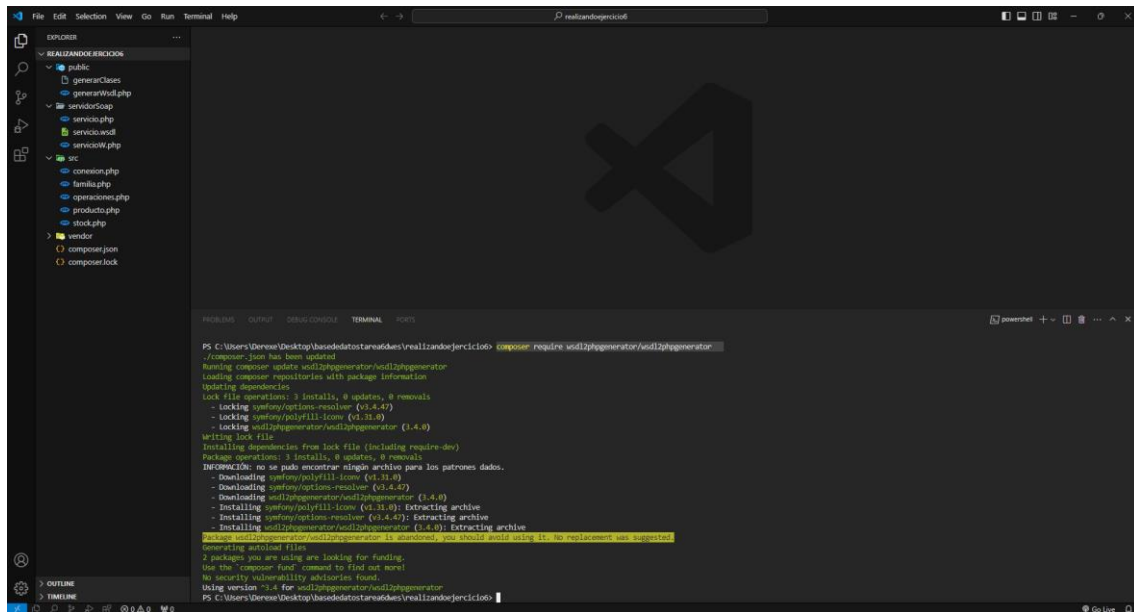
Desde el terminal CMD podemos verificar su activación con:

```
C:\Users\Derexe>php -m | findstr soap
soap

C:\Users\Derexe>
```

Como puede verse, aparece: soap, lo que indica que la extensión se ha habilitado correctamente.

Volvemos a intentar instalar la dependencia **composer require wsdl2phpgenerator/wsdl2phpgenerator**:



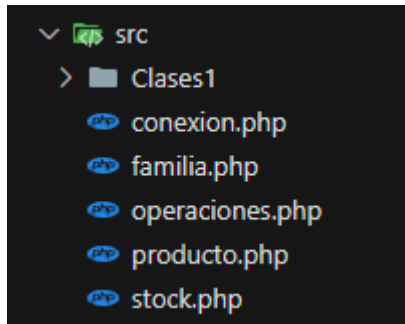
Configuramos el futuro archivo autoload.php, para ello modificamos el archivo **composer.json** de la siguiente manera:



Lo guardamos y ejecutamos desde el terminal de VSC el comando: **composer dump-autoload**.

Con esto se logrará generar el archivo autoload.php, este archivo **require** **'../vendor/autoload.php'** en lugar de usar **require** manualmente en cada archivo.

Creamos las clases en la carpeta src



Código de **conexion.php**:

```
1 <?php
2 namespace Clases;
3
4 use PDOException;
5
6 class Conexion {
7     public static function conectar() {
8         try {
9             return new PDO("mysql:host=localhost;dbname=tarea6", "gestor", "secreto", [
10                 PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
11                 PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC
12             ]);
13         } catch (PDOException $e) {
14             die("Error de conexión: " . $e->getMessage());
15         }
16     }
17 }
```

Esta clase establecerá la conexión con la base de datos.

Código de **operaciones.php**:

```

1  <?php
2  namespace Clases;
3
4  class Operaciones {
5      public function getPVP($id) {
6          return Producto::getPVP($id);
7      }
8
9      public function getStock($producto, $tienda) {
10         return Stock::getStock($producto, $tienda);
11     }
12
13     public function getFamilias() {
14         try {
15             $pdo = new \PDO("mysql:host=localhost;dbname=tarea6", "gestor", "secreto", [
16                 \PDO::ATTR_ERRMODE => \PDO::ERRMODE_EXCEPTION,
17                 \PDO::ATTR_DEFAULT_FETCH_MODE => \PDO::FETCH_ASSOC
18             ]);
19
20             $stmt = $pdo->query("SELECT cod FROM familias");
21
22             $resultado = $stmt->fetchAll();
23
24             if (!empty($resultado)) {
25                 return array_column($resultado, 'cod');
26             } else {
27                 return ["No hay datos en familias"];
28             }
29         } catch (\PDOException $e) {
30             return ["Error de conexión o consulta: " . $e->getMessage()];
31         }
32     }
33
34     public function getProductosFamilia($codFamilia) {
35         try {
36             $pdo = new \PDO("mysql:host=localhost;dbname=tarea6", "gestor", "secreto", [
37                 \PDO::ATTR_ERRMODE => \PDO::ERRMODE_EXCEPTION,
38                 \PDO::ATTR_DEFAULT_FETCH_MODE => \PDO::FETCH_ASSOC
39             ]);
40
41             $stmt = $pdo->prepare("SELECT id FROM productos WHERE familia = :codFamilia");
42             $stmt->bindParam(':codFamilia', $codFamilia, \PDO::PARAM_STR);
43             $stmt->execute();
44
45             $resultado = $stmt->fetchAll();
46
47             if (!empty($resultado)) {
48                 return array_column($resultado, 'id');
49             } else {
50                 return ["No hay productos para la familia especificada"];
51             }
52         } catch (\PDOException $e) {
53             return ["Error de conexión o consulta: " . $e->getMessage()];
54         }
55     }
56 }

```

Código de **producto.php**:

```

1 <?php
2 namespace Clases;
3
4 class Producto {
5     public static function getPVP($id) {
6         $conexion = Conexion::conectar();
7         $stmt = $conexion->prepare("SELECT pvp FROM productos WHERE id = ?");
8         $stmt->execute([$id]);
9         return $stmt->fetchColumn();
10    }
11 }

```

Código de **familia.php**:

```

1 <?php
2 namespace Clases;
3
4 class Familia {
5     public static function getFamilias() {
6         try {
7             $conexion = Conexion::conectar();
8             $stmt = $conexion->query("SELECT cod FROM familias");
9             $familias = $stmt->fetchAll(PDO::FETCH_COLUMN);
10            if ($familias === false) {
11                throw new \Exception("Error en la consulta de familias.");
12            }
13            return $familias;
14        } catch (\Exception $e) {
15            die("Error en getFamilias: " . $e->getMessage());
16        }
17    }
18 }

```

Código de **stock.php**:

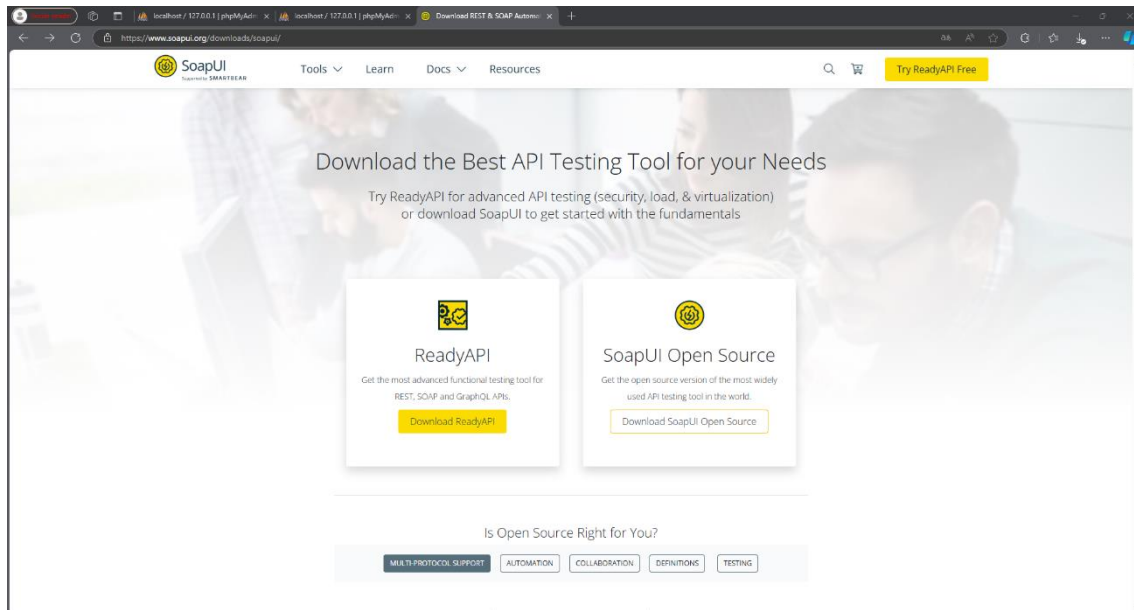
```
1 <?php
2 namespace Clases;
3
4 class Stock {
5     public static function getStock($producto, $tienda) {
6         $conexion = Conexion::conectar();
7         $stmt = $conexion->prepare("SELECT unidades FROM stocks WHERE producto = ? AND tienda = ?");
8         $stmt->execute([$producto, $tienda]);
9         return $stmt->fetchColumn();
10    }
11 }
```

Código de **cliente.php**:

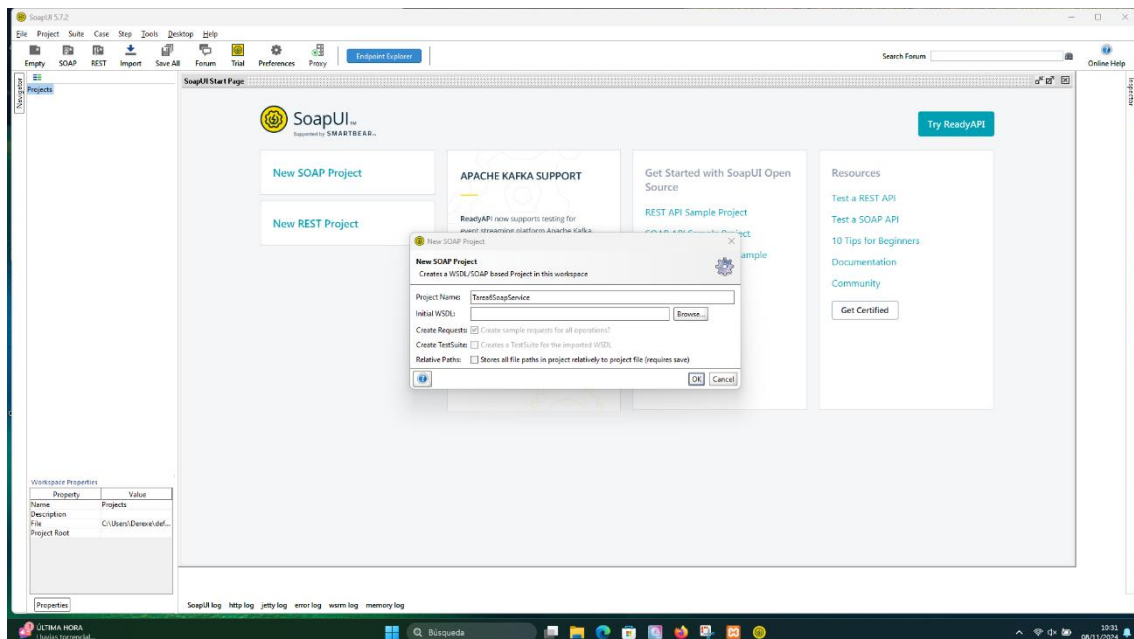
```
1 <?php
2 $url = "http://localhost/unidad6/servidorSoap/servicio.php";
3
4 $cliente = new SoapClient(null, [
5     'location' => $url,
6     'uri' => "urn:localhost/unidad6/servidorSoap",
7     'trace' => true,
8 ]);
9
10 // Llamamiento a cada función par mostrar el resultado
11
12 // getPVP
13 echo "PVP de un producto:\n";
14 try {
15     $pvp = $cliente->getPVP("PROD001");
16     echo "PVP: " . $pvp . "\n";
17 } catch (Exception $e) {
18     echo "Error en getPVP: " . $e->getMessage() . "\n";
19 }
20
21 // getStock
22 echo "Stock de un producto en una tienda:\n";
23 try {
24     $stock = $cliente->getStock("PROD001", "TIENDA1");
25     echo "Stock: " . $stock . "\n";
26 } catch (Exception $e) {
27     echo "Error en getStock: " . $e->getMessage() . "\n";
28 }
29
30 // getFamilias
31 echo "Lista de familias:\n";
32 try {
33     $familias = $cliente->getFamilias();
34     print_r($familias);
35 } catch (Exception $e) {
36     echo "Error en getFamilias: " . $e->getMessage() . "\n";
37 }
38
39 // getProductosFamilia
40 echo "Productos de una familia:\n";
41 try {
42     $productos = $cliente->getProductosFamilia("FAM001");
43     print_r($productos);
44 } catch (Exception $e) {
45     echo "Error en getProductosFamilia: " . $e->getMessage() . "\n";
46 }
```

Realizando las pruebas con SoapUI 5.7.2 SIN WSDL

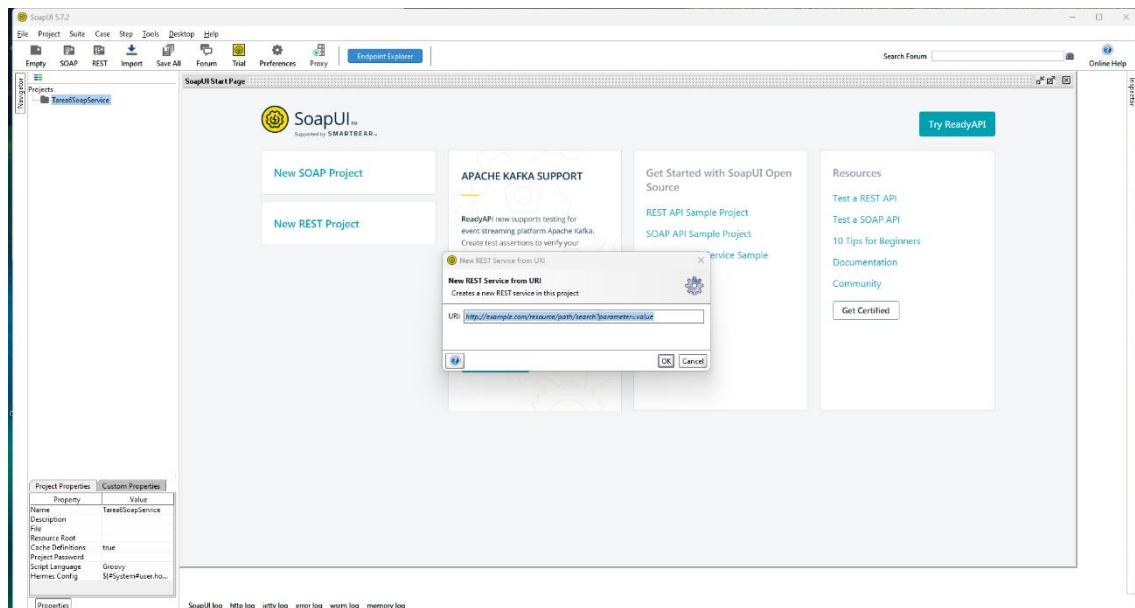
Descargar e instalar Soap desde la [web oficial](https://www.soapui.org/downloads/soapui/).



Creamos un nuevo proyecto para esta tarea sen SoapUI:



Como esta vez va a ser sin WSDL, no incluimos nada en “Initial WSDL”.

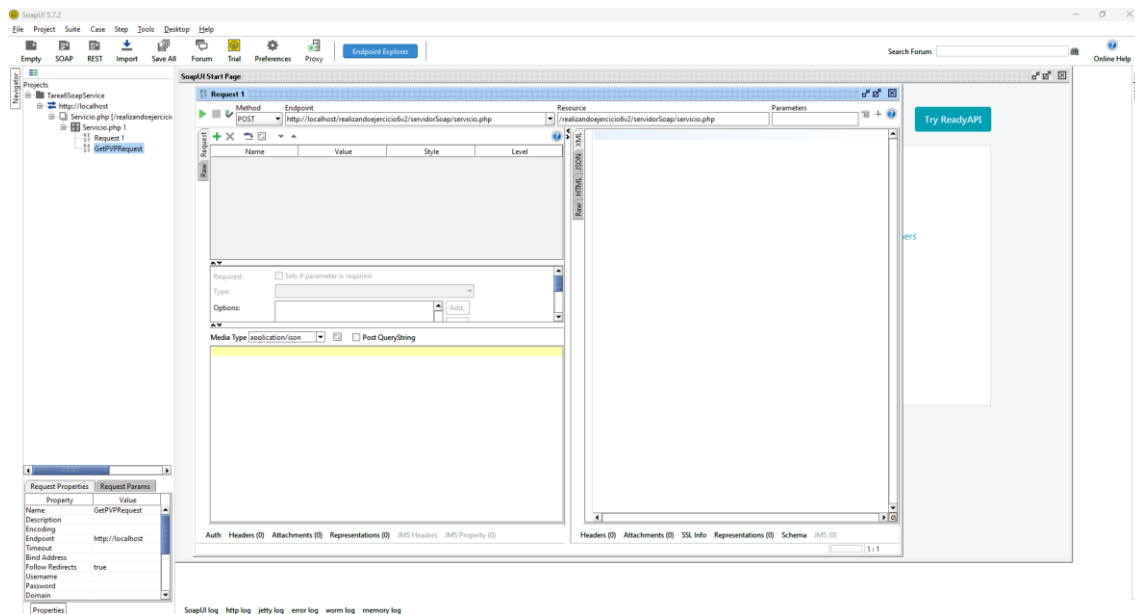


En URI pondremos (en mi caso):

<http://localhost/realizandoejercicio6v2/servidorSoap/servicio.php>

Es decir, la ubicación del archivo servicio.php.

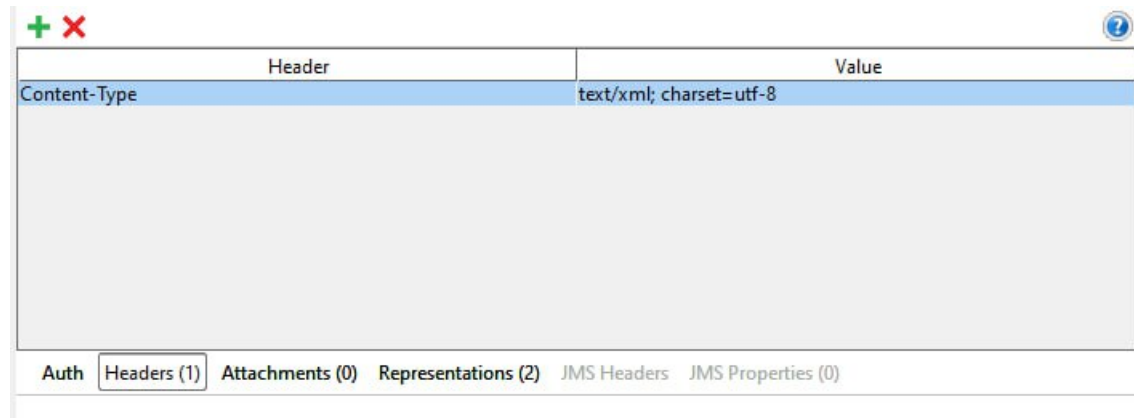
Creamos una nueva solicitud de forma manual, para esto, hacemos clic derecho sobre el proyecto y seleccionamos “Add New Request”, como nombre le ponemos “GetPVPRequest” y también con POST y en el campo Endpoint seleccionamos la ruta de **servicio.php**



En la parte inferior añadimos los headers (encabezados HTTP para que SoapUI envíe la solicitud en formato SOAP) a cada una de las futuras request que haremos, empezando por la actual: GetPVPRequest. Pulsando +.

Nombre: Content-Type (por poner un ejemplo).

Valor: text/xml; charset=utf-8



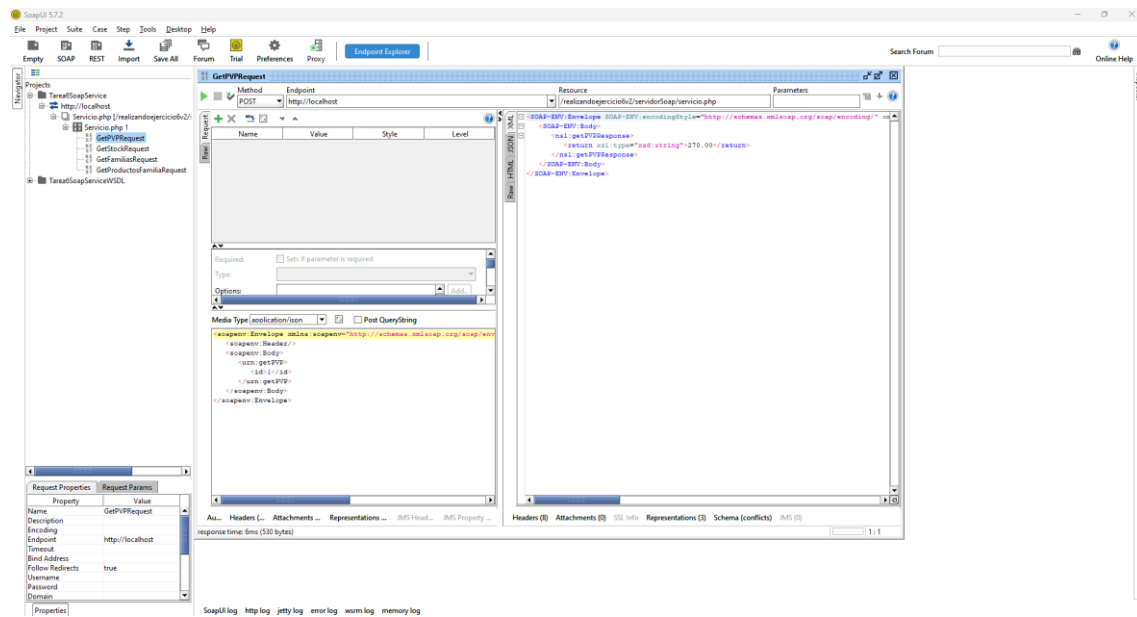
Esto debemos hacerlo antes de escribir el XML de la solicitud para cada request.

Solicitud XML para GetPVPRequest

```
<soapenv:Envelope
xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
xmlns:urn="urn:localhost/realizandoejercicio6v2/servidorSoap">
  <soapenv:Header/>
  <soapenv:Body>
    <urn:getPVP>
      <id>1</id>
    </urn:getPVP>
  </soapenv:Body>
</soapenv:Envelope>
```

Donde <id>1</id> es el número de identificación del producto que queremos consultar en la base de datos.

Mostrando la respuesta

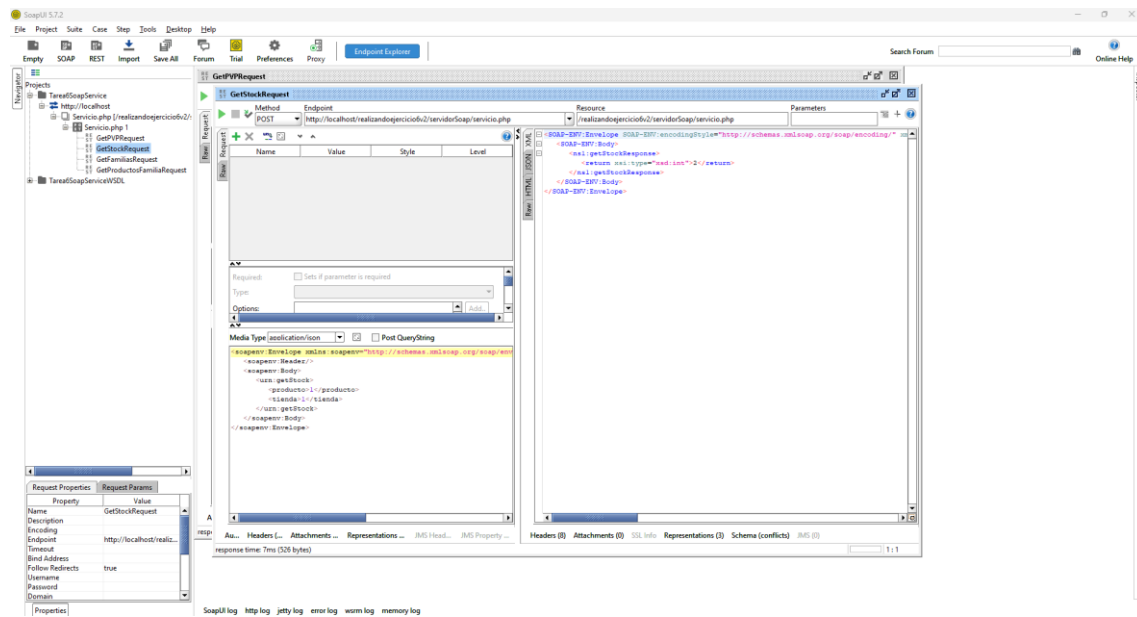


Muestra como resultado 270.00, como puede verse en la ventana de la derecha.

Solicitud XML para GetStockRequest

```
<soapenv:Envelope
xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
xmlns:urn="urn://localhost/realizandoejercicio6v2/servidorSoap">
  <soapenv:Header/>
  <soapenv:Body>
    <urn:getStock>
      <producto>1</producto>
      <tienda>1</tienda>
    </urn:getStock>
  </soapenv:Body>
</soapenv:Envelope>
```

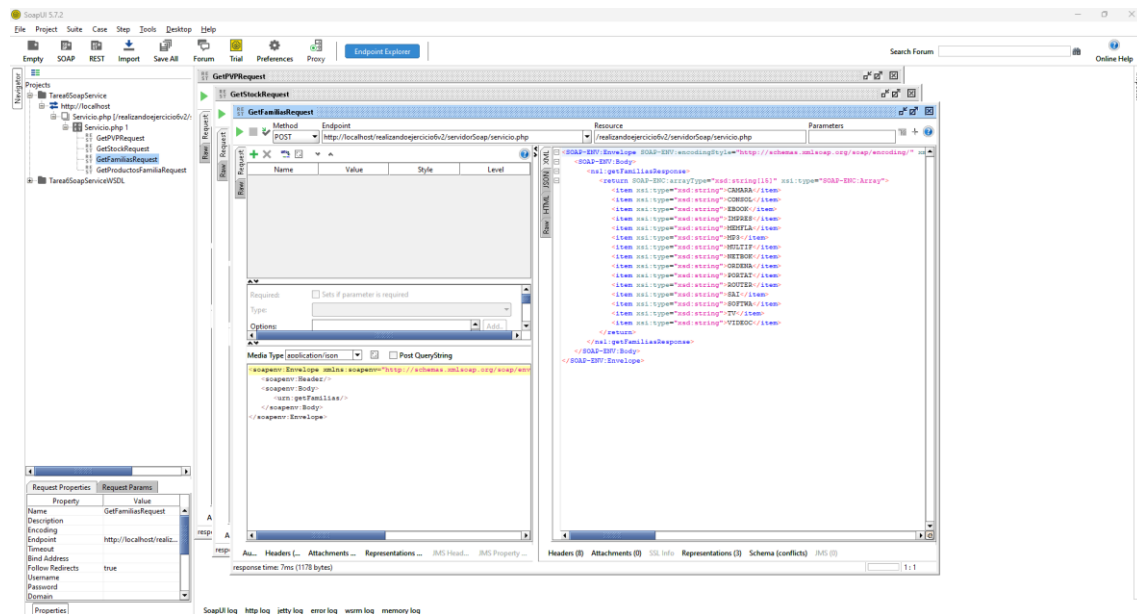
Donde <producto>1</producto> es el número de identificación del producto que queremos consultar en la base de datos, y <tienda>1</tienda> es la tienda en la que se va a buscar el stock del producto.



Muestra como resultado 2 unidades del producto, como puede verse en la ventana de la derecha.

Solicitud XML para GetFamiliRequest

```
<soapenv:Envelope
xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
xmlns:urn="urn:localhost/realizandoejercicio6v2/servidorSoap">
  <soapenv:Header/>
  <soapenv:Body>
    <urn:getFamilias/>
  </soapenv:Body>
</soapenv:Envelope>
```

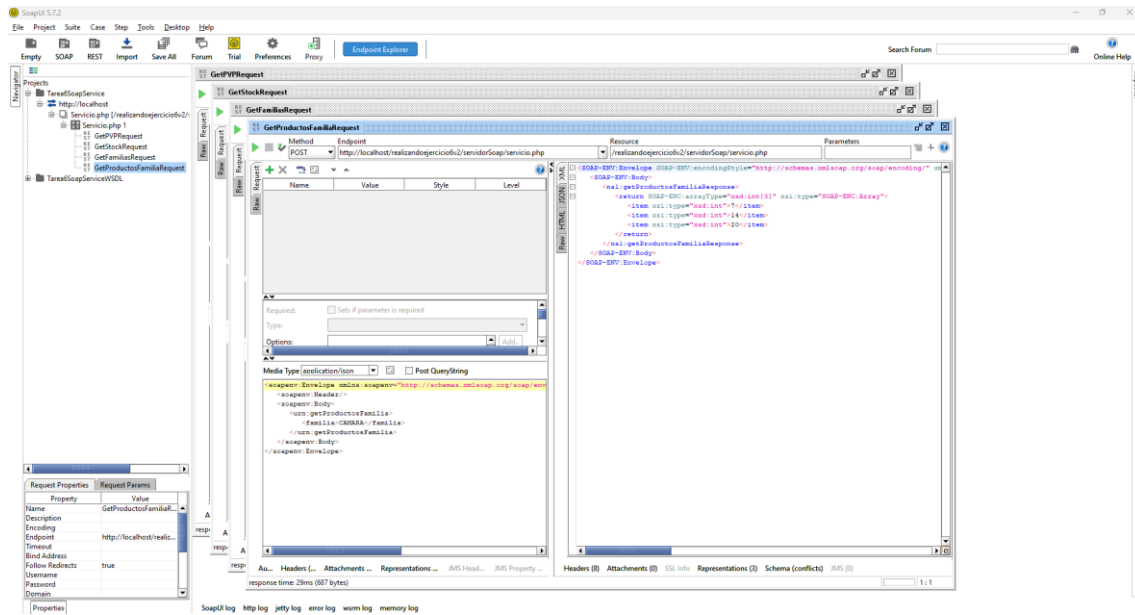


Mostrar todas las familias de productos existentes.

Solicitud XML para GetProductosFamiliasRequest

```
<soapenv:Envelope
xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
xmlns:urn="urn:localhost/realizandoejercicio6v2/servidorSoap">
  <soapenv:Header/>
  <soapenv:Body>
    <urn:getProductosFamilia>
      <familia>CAMARA</familia>
    </urn:getProductosFamilia>
  </soapenv:Body>
</soapenv:Envelope>
```

<familia>CAMARA</familia>, aquí situamos el grupo o familia de productos que estamos buscando.



Mostrar los ids de los producto de la familia que estamos buscando, en este caso CAMARA.

Realizando las pruebas con SoapUI 5.7.2 CON WSDL

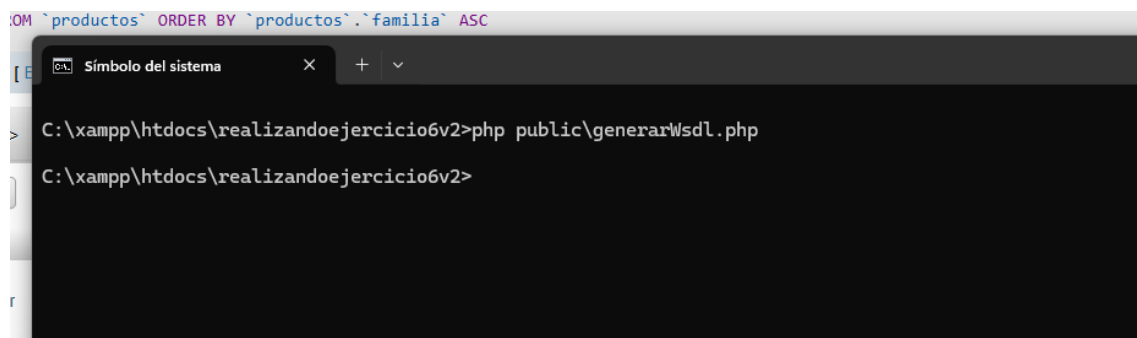
Estos son los pasos para este apartado:

Creación del archivo **servicio.wsdl**, para ello tenemos que crear el script de generarWsdL.php en la carpeta public, este archivo creará el archivo WSDL utilizando para ello la librería que instalamos anteriormente con Composer.

generarWsdL.php:

```
1 <?php
2 require __DIR__ . '/../vendor/autoload.php';
3
4
5 use PHP2WSDL\PHPClass2WSDL;
6
7 $namespace = "http://localhost/realizandoejercicio6v2/servidorSoap";
8 $class = "Clases\Operaciones";
9
10 $wsdlGenerator = new PHPClass2WSDL($class, $namespace);
11
12 $wsdlGenerator->generateWSDL();
13
14 file_put_contents(__DIR__ . '/../servidorSoap/servicio.wsdl', $wsdlGenerator->dump());
```

Ahora procedemos a ejecutar el script anterior desde el terminal:



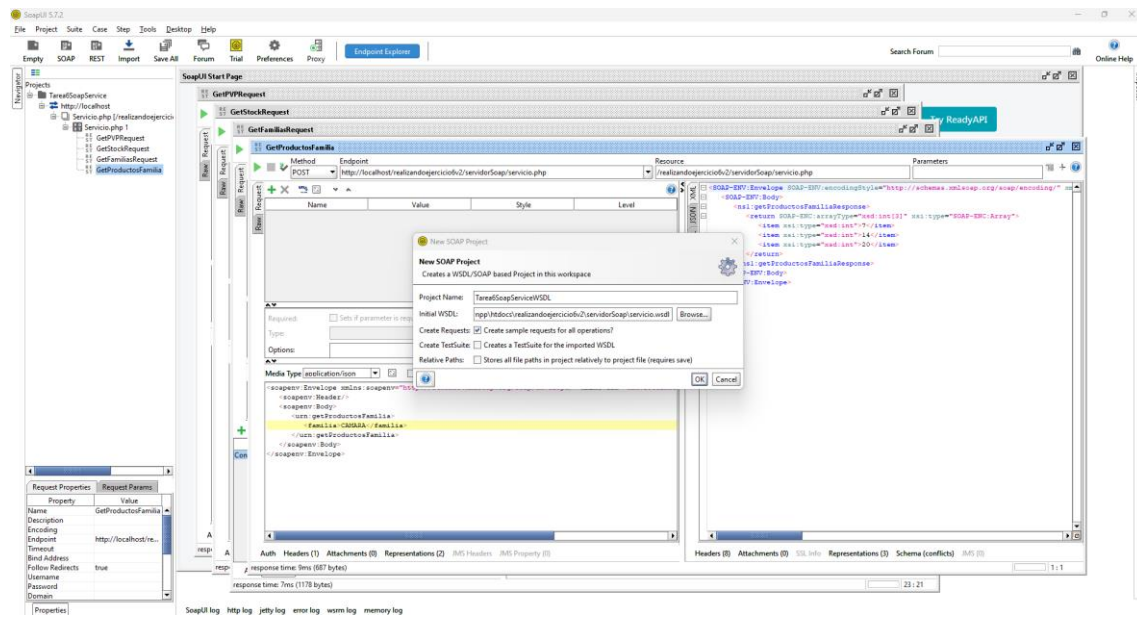
The screenshot shows a terminal window with a dark background. At the top, there's a title bar with a search icon and the text "Símbolo del sistema". Below the title bar, the terminal shows the command `C:\xampp\htdocs\realizandoejercicio6v2>php public\generarWsdL.php` being executed. The prompt then changes to `C:\xampp\htdocs\realizandoejercicio6v2>`, indicating the command was successful.

Como se puede ver, no ha generado errores, por lo que en la carpeta **servidorSoap** estará ya el nuevo archivo generado **servicio.wsdl**.

Creamos el archivo **servicioW.php** en la carpeta **servidorSoap**, y lo configuramos para que utilice el archivo generado anteriormente, **servicio.wsdl**.

```
1 <?php
2 require_once '../vendor/autoload.php';
3 use Clases\Operaciones;
4
5 $options = [
6     'uri' => 'http://localhost/realizandoejercicio6v2/servidorSoap',
7     'location' => 'http://localhost/realizandoejercicio6v2/servidorSoap/servicioW.php'
8 ];
9
10 $server = new SoapServer("servicio.wsdl", $options);
11 $server->setClass(Operaciones::class);
12 $server->handle();
```

Crear un nuevo proyecto para usar WSDL, esta vez si apuntamos desde “Initial WSDL” al archivo servicio.wsdl:

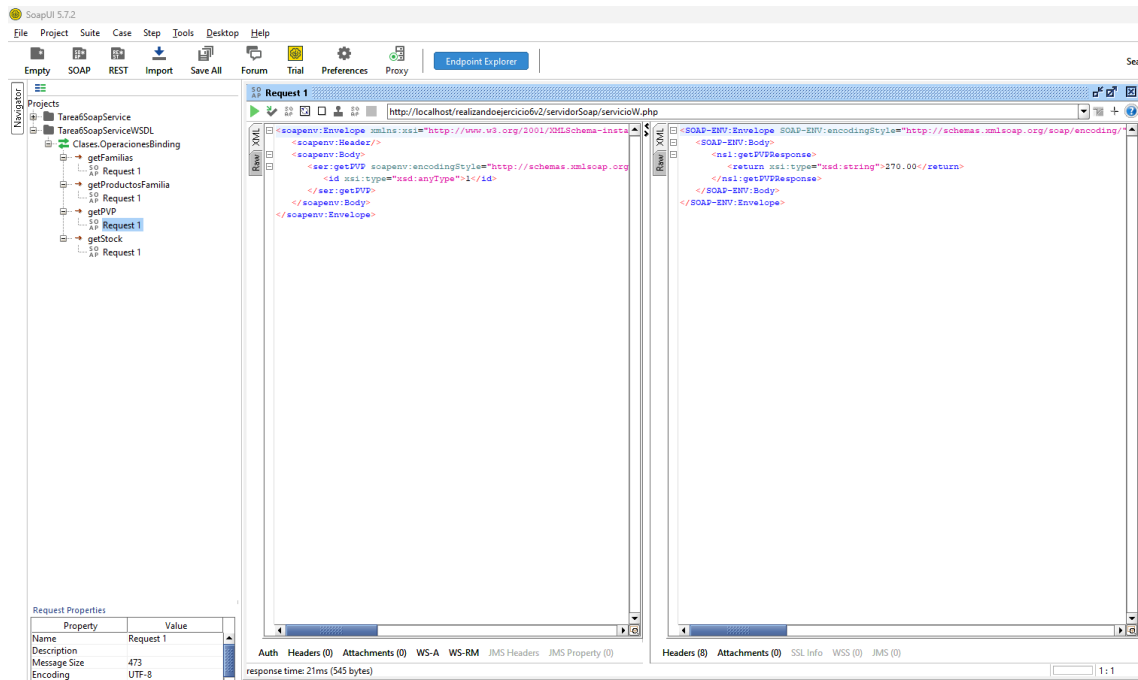


Solicitud XML para GetPVP

```

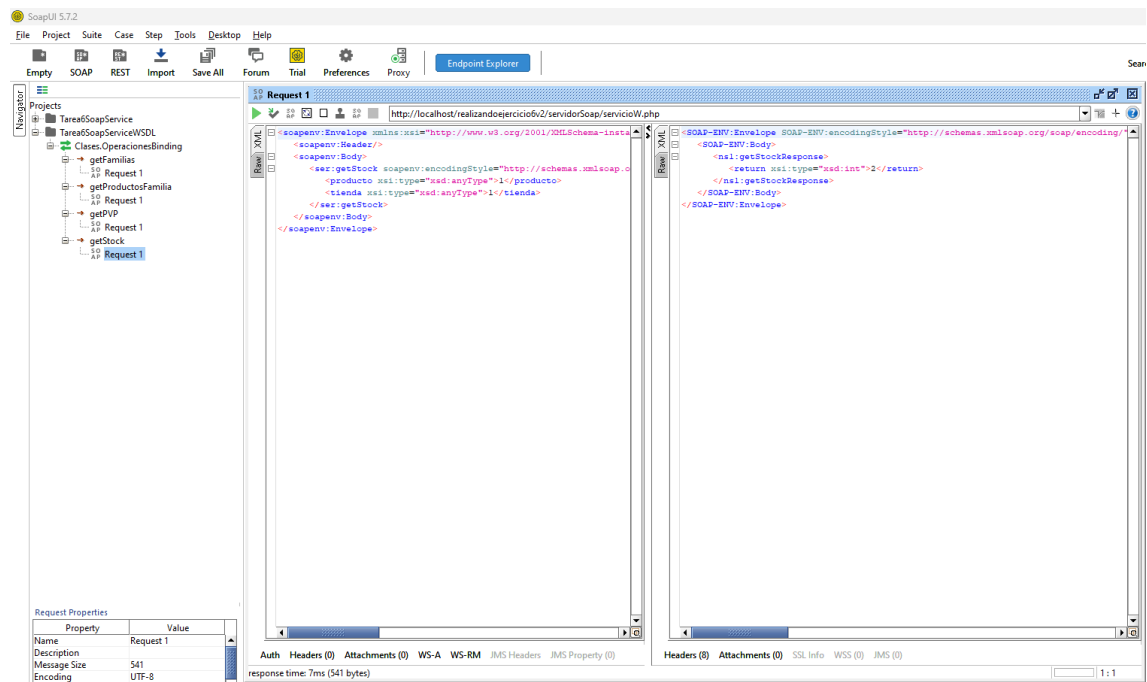
<soapenv:Envelope xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:xsd="http://www.w3.org/2001/XMLSchema"
xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
xmlns:ser="http://localhost/realizandoejercicio6v2/servidorSoap">
  <soapenv:Header/>
  <soapenv:Body>
    <ser:getPVP soapenv:encodingStyle="http://schemas.xmlsoap.org/soap/encoding/">
      <id xsi:type="xsd:anyType">1</id>
    </ser:getPVP>
  </soapenv:Body>
</soapenv:Envelope>

```



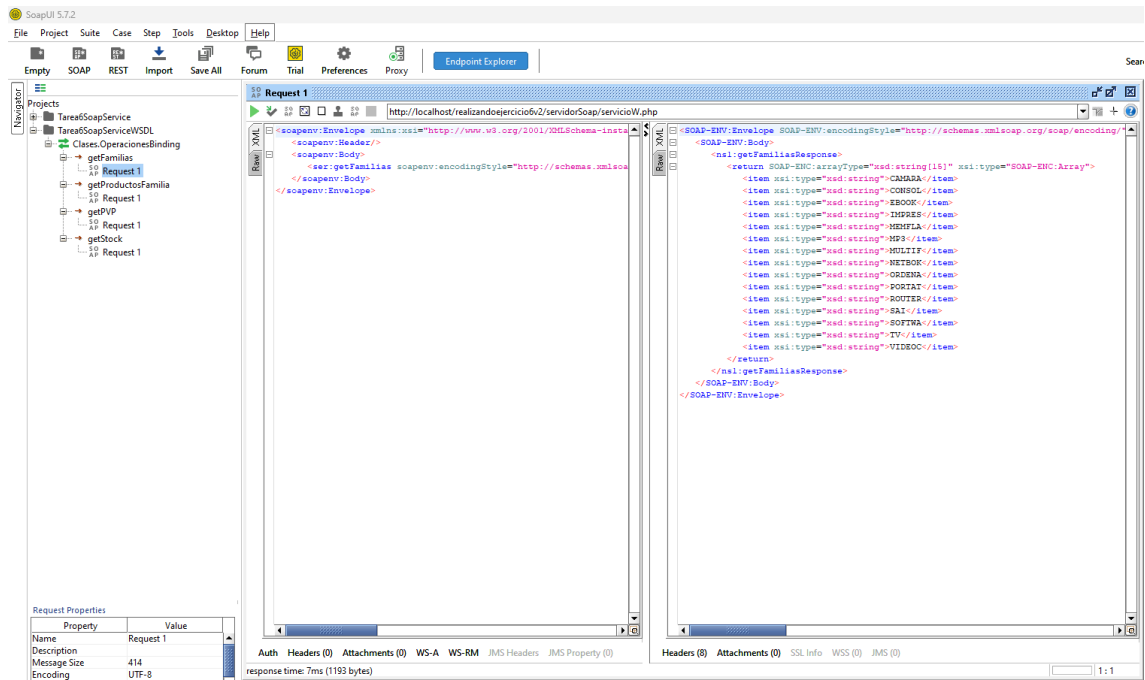
Solicitud XML para GetStock

```
<soapenv:Envelope xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:xsd="http://www.w3.org/2001/XMLSchema"
xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
xmlns:ser="http://localhost/realizandoejercicio6v2/servidorSoap">
  <soapenv:Header/>
  <soapenv:Body>
    <ser:getStock soapenv:encodingStyle="http://schemas.xmlsoap.org/soap/encoding/">
      <producto xsi:type="xsd:anyType">1</producto>
      <tienda xsi:type="xsd:anyType">1</tienda>
    </ser:getStock>
  </soapenv:Body>
</soapenv:Envelope>
```



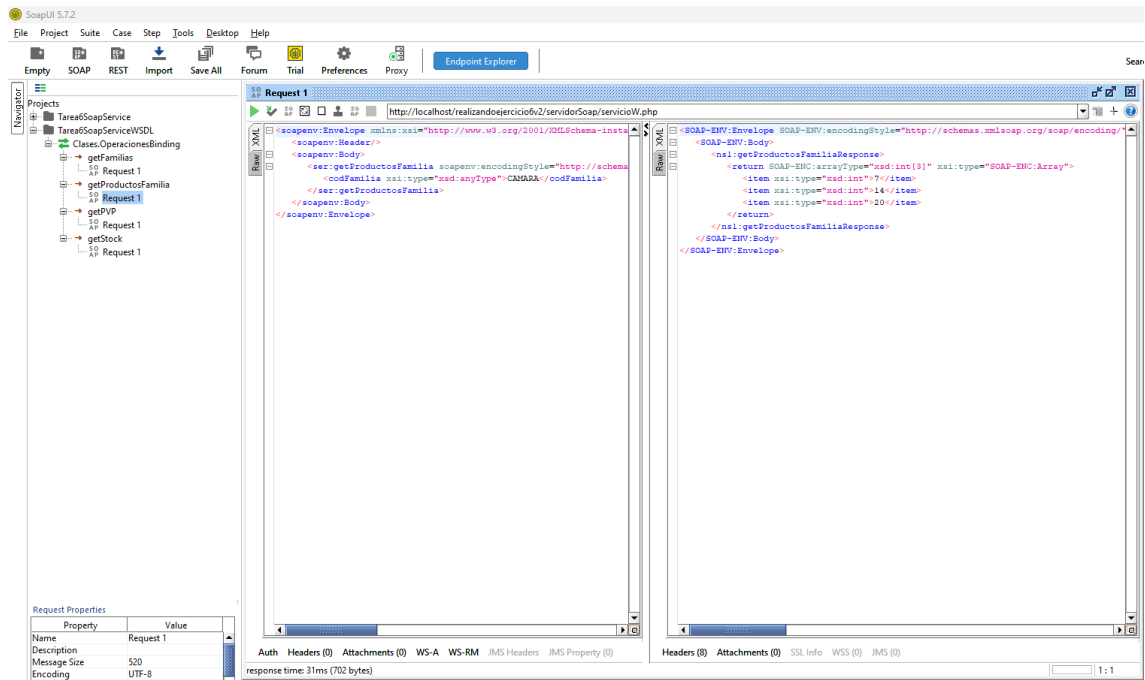
Solicitud XML para GetFamilies

```
<soapenv:Envelope xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:xsd="http://www.w3.org/2001/XMLSchema"
xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
xmlns:ser="http://localhost/realizandoejercicio6v2/servidorSoap">
  <soapenv:Header/>
  <soapenv:Body>
    <ser:getFamilies
soapenv:encodingStyle="http://schemas.xmlsoap.org/soap/encoding/">
      </soapenv:Body>
</soapenv:Envelope>
```



Solicitud XML para GetProductosFamilia

```
<soapenv:Envelope xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:xsd="http://www.w3.org/2001/XMLSchema"
xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
xmlns:ser="http://localhost/realizandoejercicio6v2/servidorSoap">
  <soapenv:Header/>
  <soapenv:Body>
    <ser:getProductosFamilia
      soapenv:encodingStyle="http://schemas.xmlsoap.org/soap/encoding/">
      <codFamilia xsi:type="xsd:anyType">CAMARA</codFamilia>
    </ser:getProductosFamilia>
  </soapenv:Body>
</soapenv:Envelope>
```



Servicio de prueba clienteW.php

Configuraremos servicioW.php para utilizar servicio.wsdl.

Contenido de clienteW.php:

```
1 <?php
2 $wsdl = 'http://localhost/realizandoejercicio6v2/servidorSoap/servicio.wsdl';
3
4 try {
5     $client = new SoapClient($wsdl);
6
7     $pvp = $client->getPVP("1");
8     echo "PVP: " . $pvp . PHP_EOL;
9
10    $stock = $client->getStock("1", "1");
11    echo "Stock: " . $stock . PHP_EOL;
12
13    $familias = $client->getFamilias();
14    echo "Familias: " . implode(", ", $familias) . PHP_EOL;
15
16    $productosFamilia = $client->getProductosFamilia("CONSOLA");
17    echo "Productos de la familia: " . implode(", ", $productosFamilia) . PHP_EOL;
18
19 } catch (SoapFault $fault) {
20     echo "Error: {$fault->getMessage()}";
21 }
```

Utilizando el comando: `php public/clienteW.php`

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\xampp\htdocs\realizandoejercicio6v2> php public/clienteW.php
PVP: 270.00
Stock: 2
Familias: CAMARA, CONSOL, EBOOK, IMPRES, MEMFLA, MP3, MULTIF, NETBOK, ORDENA, PORTAT, ROUTER, SAI, SOFTWA, TV, VIDEOC
Productos de la familia: 1, 19
PS C:\xampp\htdocs\realizandoejercicio6v2> 
```

Generación automática de clases

Utilizando wsdl2php, generaremos las clases a partir de WSDL. Ya disponemos de servicio.wsdl, por lo que la herramienta wsdl2php generará automáticamente las clases php para lograr conectar SoapUI con el servicio.

La ventaja de este sistema es que no hay que hacerlo manualmente, es mas rápido y evita posibles errores.

Al inicio del informe se indicó e instaló wsdl2phpgenerator, así que en este punto, ese apartado ya debería estar resuelto.

En la carpeta public se creará el archivo generarClases.php que es el que utilizará wsdl2php para generar las clases a partir del archivo servicio.wsdl.

Código en generarClases.php:

```
1 <?php
2 require_once __DIR__ . '/../vendor/autoload.php';
3
4 use Wsdl2PhpGenerator\Generator;
5 use Wsdl2PhpGenerator\Config;
6
7 $generator = new Generator();
8 $config = new Config([
9     'inputFile' => 'C:/xampp/htdocs/realizandoejercicio6v2/servidorSoap/servicio.wsdl',
10    'outputDir' => 'C:/xampp/htdocs/realizandoejercicio6v2/src/Clases1',
11    'namespaceName' => 'Clases1'
12]);
13
14 $generator->generate($config);
15
16 echo "Clases generadas en la carpeta 'src/Clases1'";
```

Ejecutamos en la consola el comando: `php public/generarClases.php`

Si, todo ha ido bien, ahora ya se habrán generado las clases en src/Clases1 en forma del archivo ClasesOperacionesService.php.

Contenido de ClasesOperacionesService.php:

```

1  <?php
2
3  namespace Clases1;
4
5  class ClasesOperacionesService extends \SoapClient
6  {
7
8      /**
9       * @var array $classmap The defined classes
10      */
11      private static $classmap = array (
12      );
13
14      /**
15       * @param array $options A array of config values
16       * @param string $wsdl The wsdl file to use
17      */
18      public function __construct(array $options = array(), $wsdl = null)
19      {
20
21          foreach (self::$classmap as $key => $value) {
22              if (!isset($options['classmap'][$key])) {
23                  $options['classmap'][$key] = $value;
24              }
25          }
26
27          $options = array_merge(array (
28              'features' => 1,
29          ), $options);
30
31          if (!$wsdl) {
32              $wsdl = 'C:/xampp/htdocs/realizandoejercicio6v2/servidorSoap/servicio.wsdl';
33          }
34
35          parent::__construct($wsdl, $options);
36
37      /**
38       * @param anyType $id
39       * @return anyType
40       */
41      public function getPVP($id)
42      {
43          return $this->__soapCall('getPVP', array($id));
44      }
45
46      /**
47       * @param anyType $producto
48       * @param anyType $tienda
49       * @return anyType
50       */
51      public function getStock($producto, $tienda)
52      {
53          return $this->__soapCall('getStock', array($producto, $tienda));
54      }
55
56      /**
57       * @return anyType
58       */
59      public function getFamilias()
60      {
61          return $this->__soapCall('getFamilias', array());
62      }
63
64      /**
65       * @param anyType $codFamilia
66       * @return anyType
67       */
68      public function getProductosFamilia($codFamilia)
69      {
70          return $this->__soapCall('getProductosFamilia', array($codFamilia));
71      }
72  }

```

Esta clase principal permite al cliente interactuar con el servicio web y contiene las operaciones: getPVP, getStock...

ClasesOperacionesService.php será utilizado para conectarse y realizar llamadas al servicio SOAP de forma directa (sin utilizar protocolos SOAP o solicitudes XML como hasta arhoa).

En el siguiente apartado (clienteW2.php) puede verse como se utiliza ClasesOperacionesService.php para conectar con el servicio (línea 2).

Servicio de prueba clienteW2.php

Contenido de clienteW2.php:

```
1 <?php
2 require_once __DIR__ . '/../src/Clases1/ClasesOperacionesService.php';
3
4 use Clases1\ClasesOperacionesService;
5
6 try {
7     $cliente = new ClasesOperacionesService();
8
9     $pvp = $cliente->getPVP(1);
10    echo "PVP del producto 1: $pvp\n";
11
12    $stock = $cliente->getStock(1, 2);
13    echo "Stock del producto 1 en la tienda 2: $stock\n";
14
15    $familias = $cliente->getFamilias();
16    echo "Familias: " . implode(", ", $familias) . "\n";
17
18    $productosFamilia = $cliente->getProductosFamilia("CAMARA");
19    echo "Productos de la familia 'CAMARA': " . implode(", ", $productosFamilia) . "\n";
20
21 } catch (Exception $e) {
22     echo "Error: " . $e->getMessage();
23 }
```

Utilizando el comando: `php public/clienteW2.php`

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS C:\xampp\htdocs\realizandoejercicio6v2> php public/clienteW2.php
PVP del producto 1: 270.00
Stock del producto 1 en la tienda 2: 1
Familias: CAMARA, CONSOL, EBOOK, IMPRES, MEMFLA, MP3, MULTIF, NETBOK, ORDENA, PORTAT, ROUTER, SAI, SOFTWA, TV, VIDEOC
Productos de la familia 'CAMARA': 7, 14, 20
PS C:\xampp\htdocs\realizandoejercicio6v2> []
```

FIN